

Basics of Programming & Introduction to C/C++

By Utkarsh Sharma

Welcome to Programming Basics

- Agenda:
 - Motivation for Programming
 - High-Level and Low-Level Languages
 - Introduction to common programming languages
 - C++ Data types and memory
 - Programming entities like variables, control structures, sequential access
 - To be able to write a simple code
- QnA

Motivation behind programming

- To be able to instruct a computer
- Programming is a process of converting ideas into instructions that a computer can understand and execute
- Instructions need to be specific and sequential

Programming Languages

- Instructions can be given in:
 - ❖ **Natural Language:** Regular English, can be highly ambiguous
 - ❖ **Machine Language:** Can directly access the processor/hardware through binary codes (0s and 1s), but very difficult to learn
 - ❖ **Assembly Language:** An abstraction of machine language, uses codes to modify processor registers and perform functions



Assembly Languages



Programming Languages – Continued

❖ Interpreted Languages:

- ❑ The programmer writes the code, gives it to the **interpreter** which interprets the code into machine code for the hardware to read, understand and execute
- ❑ Execution can be stopped; a line of code can be updated, and the execution can be resumed without any other steps
- ❑ For example, Python, JavaScript



Programming Languages – Continued

❖ Compiled Languages:

- ❑ The programmer writes the code, gives it to a **compiler** which converts the code into native code specific to that hardware and OS and creates a program (compiles) called *an executable*
- ❑ For any code updates, the *executable* needs to be re-compiled
- ❑ For example, C/C++



Programming Languages – For Data

- ❖ **SQL**, pronounced as sequel:
 - You can ask a question to a database, and it returns a dataset as an answer
 - Databases can also be combined based on various conditions for analysis
- ❖ **R**
 - Mainly used for statistical computing and graphing

Did everyone understand the things so far?

Question: What is the difference between an interpreted and a compiled programming language?

- A. An interpreted language has each line converted to native instructions as it is run. Compiled languages convert everything to an executable at once.
- B. Interpreted languages are used for web-based applications. Compiled languages are for apps like iOS, Android, and Windows.
- C. Interpreted languages are a previous generation of languages before the creation of compiled languages.
- D. Interpreted languages are based on transformations of values. Compiled languages are created using step-by-step algorithms.

Variables and Data-types

- Computers store, process and access data using variables
- Variables have a name (used by the programmer)
- Get a reserved space in the memory to be used by the computer
- Each variable has a data type, defining what kind of data that variable holds
- Various data types like integers (signed & unsigned), decimals, characters and strings

Variables and Data-types

- Boolean (bool): 1 byte
- Characters (char): 1 byte
- Integers (int): 4 bytes
- Decimals (float): 4 bytes
- Double (double the decimal precision): 8 bytes

Declaring Variables in C++

- The basic syntax is:

`<data type> <variable name> = <initial value>;`

Or

`<data type> <variable name>;`

- For example:

```
1  int a = 10;  
2  float myFavoriteNumber = 3.1415;
```

Quiz on Data Types

Q1

How do you create a variable with the floating number 2.8?

- ☐ byte x = 2.8
- ☐ double x = 2.8;
- ☐ int x = 2.8;
- ☐ x = 2.8;

Q2

How do you create a variable with the numeric value 5?

- ☐ x = 5;
- ☐ num x = 5
- ☐ int x = 5;
- ☐ double x = 5;

FUNCTIONS in programming

- Function is declared and then defined (just like variables)
- A set of statements that take some input, does some told computations and **returns** an output
- Declaration syntax is:

`<return type> <function name> ( <input var 1>, <input var 2>, ... , <input n>);`

- For example:

```
1  ✓ int addThreeNumbers(int a, int b, int c)
2    {
3    |    //definition block
4    }
```

Defining a Function

- A function's definition includes the computations we want it to perform
- Definition is always bounded by '{' and '}'. This {} indicates the *scope* of function.
- The function in the previous slide addThreeNumbers can be defined as:

```
1  int addThreeNumbers(int a, int b, int c)
2  {
3      int sum = a + b + c;
4      return sum;
5  }
```

Necessary lines of code

- We use **header** files, which include function definitions and data type definitions
- Things that will always appear in your code:

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  ✓ int main()
4  {
5  |    //execution of a file starts here
6  }
```

Input Output

- To print something (**output**), we use ‘**cout**’:

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main()
4 {
5     float myFavoriteNumber = 3.1415;
6     cout << "whatever that needs to be printed, can be a variable name";
7     cout << endl;
8     cout << myFavoriteNumber ;
9 }
```

OUTPUT

```
whatever that needs to be printed, can be a variable name
3.1415
```

- To get something from the user (**input**), we use ‘**cin**’ :

```
int main()
{
    int a;
    cout << "Enter a number" << endl;
    cin >> a;
    cout << "The number you entered is : " << a;
}
```

OUTPUT

```
Enter a number
21
The number you entered is : 21
```


Our first C++ program: Adding 2 numbers

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main(){
4      int a = 10;
5      int b = 20;
6      int sum = a + b;
7      cout<<"Sum of the numbers is :: " << sum;
8  }
```

With fixed values of variables

OUTPUT

Sum of the numbers is :: 30

```
3  int main()
4  {
5      int a, b;
6      cin >> a >> b;
7      int sum = a + b;
8      cout<<"Sum of the numbers is :: " << sum;
9  }
```

With user-given values of variables

OUTPUT

10
25
Sum of the numbers is :: 35

Increment and Decrement

```
1  #include<bits/stdc++.h>
2  using namespace std;
3  int main(){
4      int a ;
5      cout << "Enter a number :: " ;
6      cin >> a ;
7      cout << "Current value of the variable is = " << a << endl;
8      a++ ;
9      cout << "Current value of the variable is = " << a << endl;
10     a-- ;
11     cout << "Current value of the variable is = " << a;
12 }
```

Enter a number :: 30

Current value of the variable is = 30

Current value of the variable is = 31

Current value of the variable is = 30

Object Oriented Programming

```
4 class class1{
5     public:
6         int a = 10;
7         string name = "xyz";
8         void printDetails(){
9             cout << "a = " << a << endl << "name = " << name << endl;
10        }
11 };
12 int main(){
13     class1 object1;
14     object1.printDetails();
15     object1.a = 20;
16     object1.name = "pqr";
17     object1.printDetails();
18 }
```

OUTPUT

```
a = 10
name = xyz
a = 20
name = pqr
```